Towards an Efficiently Simulated Quantum Approximate Optimisation

Amir Alizadeh, Amir Pourabdollah and Ahmad Lotfi
Department of Computer Science, Nottingham Trent University, UK
amir.alizadeh|amir.pourabdollah|ahmad.lotfi@ntu.ac.uk

2025 IEEE International Conference on Fuzzy Systems (FUZZ) | 979-8-3315-4319-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/FUZZ62266.2025.11152265

***Abstract*—Many large-scale optimisation problems, such as those in power systems, finance and logistics can be potentially solved in the fundamentally new approach of quantum algorithms, namely Quantum Approximate Optimisation Algorithm (QAOA). In the context of fuzzy systems for instance, it is already shown that some of the intense computation associated with complex or higher-order fuzzy logic systems can be offloaded to such quantum algorithms. While the quantum computation hardware technology is still in its infancy, efficient simulations of the quantum algorithms play a crucial role in advancing quantum algorithms. The current CPU/GPU-based approaches suffer from high energy consumption and speed and scalability limitations. This paper presents the current position of our research to develop an efficient and scalable QAOA simulation based on a large network of Field-Programmable Gate Array (FPGA) modules, inspired by FPGA's hardware-level parallelism. Although there exist a few attempts on using FPGA for general quantum circuits simulation, this project introduces a novel approach by employing a highly efficient recursive matrix decomposition algorithms specifically designed for QAOA. This work can pave the pathway a scalable and energy-efficient solution for cross-domain optimisation problems. The actual results and evaluations are to be reported in follow-up publications.**

***Index Terms*—quantum computing, optimisation, QAOA, FPGA**

I. Introduction

Over the past decades, developments in Quantum Computation (QC) opened new opportunities to tackle complex problems, specifically in the field of optimisation problems, such as Combinatorial Optimisation Problems (COPs), Max-Cut, Max-SAT, and other NP-hard problems to which many real-world optimisation scenarios can be reduced. Accordingly, Quantum Approximate Optimisation Algorithm (QAOA) [1] has emerged as a competitive candidate for solving COPs. This approach demonstrates potential efficiency compared to classical computation by combining quantum mechanical principles with classical machine learning algorithms.

Briefly, QAOA bridges between two major quantum computing paradigms: circuit-based and annealing methods. This is based on approximating a quantum annealing process by cascading tunable quantum circuits, in which the tuning is realised by classical machine learning methods. Given that quantum annealing is specialised to minimising polynomials in Binary Quadratic Form (BQF), any optimisation problem that can be reduced to minimising BQF (both constrained or unconstrained) can be tackled by QAOA.

Apart from some known industrial applications (such as in power systems [2]), the fuzzy logic community has also recently highlighted the potential contribution of quantum algorithms in fuzzy systems, particularly in complex and higher-order systems when the intensive calculations become the bottlenecks. There now exist quantum counterparts for fuzzy logic's basic operations [3], for fuzzy inference engines [4], [5], and for complex type-2 defuzzification methods such as Karnik-Mendel's algorithm [6]. Many of these methods involve mapping the required calculations to solving a BQF optimisation problem, which is then solved either directly by a quantum annealer or approximately by QAOA.

Although QAOA is designed for near-term quantum hardware, its simulation on classical hardware remains essential for verifying correctness, exploring algorithmic variations, assessing quantum advantages, and performance profiling. Research results show that the QAOA algorithm has the potential to outperformance the state-of-the-art classical solver's performance in complex COPs, which has motivated significant effort to achieve large-scale QAOA simulation in the absence of large-scale fault-tolerant quantum hardware [7] [8]. However, simulating quantum algorithms on classical hardware becomes computationally expensive as the number of qubits grows - leading to very large matrix multiplications. The current state-of-the-art approaches utilise CPUs and GPUs to parallelise matrix-vector multiplications or tensor network construction. Although GPUs, by leveraging parallelism, provide much more efficient computation over CPUs for certain quantum circuits, they are still constrained by power consumption, memory, bandwidth, and parallelisation efficiency.

Field-Programmable Gate Arrays (FPGAs) have already gained attention in high-performance computing for their ability to become customised architectures, efficient resource utilisation, and low-latency pipelines [9]. These features make FPGAs a compelling platform to explore for quantum circuit simulation, where massive parallelism, precision, and large memory bandwidth are critical. Using FPGA for quantum computing has recently been explored in a few research works [10]–[13]. However, the research gap lies in scalability and efficiency in resource utilisation, particularly on multiple FPGA modules. We have also been inspired by the recent developments in efficient GPU-based QAOA simulation (see Section II), which can potentially be mapped into FPGA-based simulation. Despite its potential, to the best of our

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 21:05:09 UTC from IEEE Xplore. Restrictions apply.

knowledge, there is no scalable FPGA-based simulation of QAOA.

This paper presents the current status of our project to implement a QAOA simulator using FPGA hardware. While the methodology reported in this paper is developed, the hardware implementation and test is under development. Specifically, the main aims are to (a) design an FPGA-based architecture to simulate QAOA focusing on the gate-by-gate evolution and state vector manipulation; (b) Compare its performance and resource utilisation with that of CPU and GPU-based simulators; and (c) Address scalability challenges by exploring optimisations in the FPGA architecture (such as fixed-point vs. floating-point arithmetic, parallel gate application, and memory management). Given that the project is still in progress and its implementation is not yet finished, this paper focuses on the motivation, literature review, methodology, architecture design, and initial findings. The key milestones achieved will be highlighted in this paper alongside the next steps to finalise the implementation.

II. Background and Related Work

A. Quantum Approximate Optimisation Algorithm (QAOA)

The QAOA, known as a variational quantum algorithm, has acquired a great deal of attention due to not only its capability to solve complex optimisation problems but also to its error resilience [1]. QAOA introduces a framework that merges quantum computation with classical methods by alternating between applying a problem Hamiltonian H_C (associated with the cost function to be minimised) and a mixing Hamiltonian H_M. The algorithm is parameterised by angles β and γ, which are tuned by a classical optimiser part to minimise the cost function in p steps, as a result of which the solution space approaches the optimal point as $p \to \infty$ [14]. This is intended to optimise real parameters repeatedly in order to approximate the solutions of COPs in the form of:

$$i\hbar \frac{d}{dt}|\psi(t)\rangle = H(t)|\psi(t)\rangle \tag{1}$$

where the Hamiltonian $H(t) = A(t)H_M + B(t)H_C$. Also, $A(t)$ and $B(t)$ are functions that very slowly transition between the two ground states. Let T be the total evolution time proportional to $O(1/\Delta E_m^2)$ where ΔE_m is the spectral gap encountered during the evolution. Thus, $A(0) = 1, B(0) = 0, A(T) = 0, B(T) = 1$. The Hamiltonian can then be:

$$H(t) = \left(1 - \frac{t}{T}\right) H_M + \frac{t}{T} H_C \tag{2}$$

By dividing this evolution time into steps $\Delta\tau$ using the Trotter-Suzuki method, the continuous adiabatic process will be discretised into a number of gate-based circuits. This technique leads to a parametrised unitary operator $U(\beta, \gamma)$ as:

$$U(\beta, \gamma) = \prod_{k=1}^{p} e^{-i\beta_k H_M}\, e^{-i\gamma_k H_C} \tag{3}$$

where $p = T/\Delta\tau$ and the real-valued β_k, γ_k are parameters represent each component's time-dependent nature (details in [15]). In general, solving a given COP using QAOA algorithm requires n-qubit register (the problem size) and two gates to prepare H_M and H_C that encode a cost function as:

$$H_M = -\sum_{i=1}^{n} X_i,\ H_C = \sum_{i,j=1}^{n} J_{jk} Z_i Z_j + \sum_{i=1}^{n} h_i Z_i \tag{4}$$

where J and h are real-valued matrix and vector, respectively. The QAOA circuit evolves the system from an initial state $|\psi_0\rangle$ to an approximate ground state H_C via $U(\beta, \gamma)$ formulated by:

$$|\psi(\beta, \gamma)^p\rangle = \prod_{k=1}^{p} e^{-i\beta_k H_M}\, e^{-i\gamma_k H_C}\, |\psi_0\rangle\,. \tag{5}$$

The energy of this quantum state can be achieved by measuring all qubits, known as the expectation value as:

$$\langle C(\beta, \gamma)^p\rangle = \langle\psi(\beta, \gamma)^p|\, H_C\, |\psi(\beta, \gamma)^p\rangle\,. \tag{6}$$

For a given COP, one would need to find the set of optimal parameters that minimise (6), e.g., by a classical optimiser.

B. Existing Hardware Platforms for QAOA Simulation

The two main current approaches for simulating the QAOA circuit on classical hardware are tensor product methods and state vector simulation algorithms. The tensor network method is known for its ability to simulate large-scale circuits, as this approach uses a QAOA circuit structure and does not need to store full-state vectors. Although this approach provides more efficient simulation in specific quantum circuits, it can be less effective than state vector evaluation when simulating QAOA circuits. Particularly, their applicability is restricted in the face of deep circuits, where the tensor network can efficiently manage only a limited number of layers [16]. It has been theoretically proved that as the number of iterations in the QAOA algorithm increases, the exploration of the solution space expands, increasing the likelihood of finding the optimal solution. However, this also causes a significant rise in the complexity of the tensor network, reducing its efficiency compared to state vector methods [17].

On the other hand, the state vector evaluation simulates the entire Hilbert space - a straightforward implementation that provides a competitive approach. However, the method is limited to storing vectors of size 2^n, where n is the number of qubits. Many efforts have been made to improve the efficiency of matrix multiplication and memory requirements to increase the number of qubits simulations in this approach [18]. Gate fusion combines multiple gates into a single larger gate as an approach to improve memory efficiency in quantum simulators [18] [19]. While typically applied to two-qubit systems, this approach significantly boosts simulation speed. However, it faces limitations due to memory requirements, as the fused gate necessitates storing 4^n complex numbers, where n is the number of qubits. Lykov et al. developed QOKit model [20] as an extreme case of gate fusion, in which the phase operator

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 21:05:09 UTC from IEEE Xplore. Restrictions apply.

in QAOA produces a diagonal gate being stored as a vector of size 2^n. They introduced a pre-computation approach for the cost vector, which represents the diagonal of the problem Hamiltonian. This precomputed cost vector is then used to efficiently apply the phase operator and to evaluate the QAOA objective. Several CPU/GPU simulation back-ends were then used to scale up simulation through parallel processing. In a similar approach, the JuliaQAOA framework [21] shared the same key features such as pre-computation and the similar implementation of the cost Hamiltonian function, written in Julia [22].

As mentioned in previous examples, the current approaches for simulating large-scale QAOA circuits utilise GPU-accelerated for parallelisation computation. These methods are mainly implemented through the cuStateVec SDK [23]. This toolkit enables the conversion of quantum circuit instruction, originally written in other frameworks such as Qiskit or Pennylane, into CUDA code. In a recent paper, the CUAOA [24] framework was introduced, in which the authors improve the QOkit framework by rewriting it in Rust and C/C++ being more compatible with CUDA than Python, to reduce time latency in simulation. This CUDA-accelerated model is developed for single GPU usage to overcome the limitation of the QOkit model by providing a more efficient and feature-complete implementation.

C. FPGA-based Quantum Computation

FPGAs are capable of massive parallelism and customisable architectures due to their flexible programming, vast array of logic functions, and low latency. Including DSP units in modern FPGAs, which provide optimised computational modules, offers further enhancement capacity. The DSP units can access data at high rates from fast, flexible in-chip memory (BRAM), enabling them to perform up to two multiplications per clock cycle. More importantly, FPGA consume much less power than CPU/GPU-based solutions. Several studies have demonstrated the potential of FPGA-based implementations in the simulation of quantum circuits [10]–[12]. For example, the reported notable speed-ups by FPGA accelerators that efficiently handle complex quantum operations compared to traditional simulation methods [13]. Despite these promising developments, a careful review of the literature reveals that none of the existing FPGA-based simulation studies have been tailored to the QAOA. Particularly, the iterative nature of QAOA and its FPGA-friendly computational characteristics motivate our current research.

D. Connection To Computational Intelligence

The main focus of this paper is quantum computing simulation, but the integration with computational intelligence methods such as fuzzy logic or neural-based optimiser significantly enhances the applicability of QAOA simulations. One of the main issues that leads to latency is communication between the simulation part and the classical optimiser. This becomes much challenging as communication between the FPGA board and global memory is expensive, so an optimiser with fewer iterations would enhance the latency. Recent advancements demonstrate that fuzzy systems and neural networks can effectively handle parameter uncertainties and optimise variational parameters (β, γ) of QAOA through adaptive strategies. Hybrid approaches, for instance, neural-based Bayesian optimisation, can dynamically adjust parameters in response to quantum noise and significantly reduce the number of calls to the quantum part [25] [26].

III. Methodology Overview

This section explores our approach to simulating QAOA on an FPGA platform efficiently. While the research remains ongoing, the initial research guided us to the development of a method to address the computational and scalability challenges inherent in simulating QAOA on a classical system. Our methodology relies on the *Trotterisation* technique to decompose the evolution time operator into manageable steps that enable efficient matrix multiplication and state vector manipulation. This method is essential for addressing the rapid growth in the state vector's size as the number of qubits increases.

A. Algorithm Strategies

By considering different strategies for simulation QAOA on FPGA hardware, we selected the state vector simulation approach for its simplicity and capability of handling deep circuits. In contrast, other methods, such as tensor network contractions, often become unwieldy for deep circuits. The state vector simulation generally consists of two functions: A mixer (phase) function H_m and a cost function. The complexity of the current simulation of mixer operation $O(n2^n)$ for n qubits is such that the additional speed would be negligible due to a fundamental constraint in the structure of this operator [20]. Thus, researchers focus more on the diagonal cost operator, where parallelisation and efficient implementation with GPU are made possible.

B. Cost Function Representation

The cost operation H_C in QAOA is implemented by using a quantum gate for modelling the optimisation cost function $f(s)$ in QAOA simulation. H_C has a diagonal structure where each element corresponds to $f(s)$ of each basic state $|s\rangle$.

$$f(s) = \Sigma^l w_k \prod_{t_k} s_i \tag{7}$$

where t_k defines subsets of variables s_i and w_k are weights.

In the classic quantum gate simulation, this operation would require a sequence of matrix multiplications on the state vector to update its elements. However, the pre-computation approach in QOkit and JuliaQAOA frameworks enhances this computation by bypassing the need for gate-base implementations. By knowing that H_c has a diagonal structure in most applications, these frameworks replace the series of gates with pre-computes $f(s)$ values for all the states. It applies H_C as element-wise (bitwise-XOR) and population count operation, which determines the sign of the product in multiplication.

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 21:05:09 UTC from IEEE Xplore. Restrictions apply.

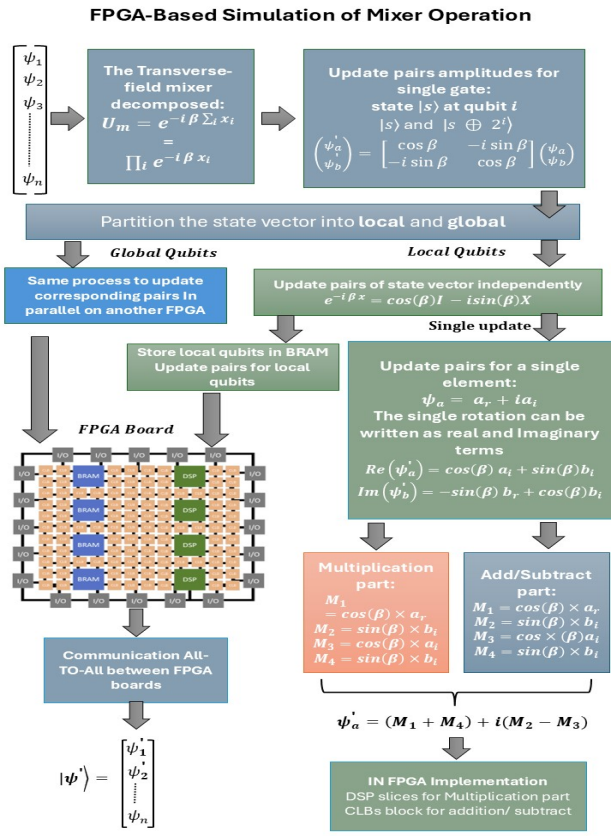

Fig. 1. An overview of simulating mixing operation on FPGA boards

To parallelise this computation and to lower the computing overhead, GPUs can be employed. Some researchers attempted to increase the GPU parallelism by rewriting QOkit in Rust, such as CUAOA [24], which is more compatible with CUDA since there is no foreseen improvement in other parts of the simulation, such as mixer layer X_m. On the other hand, FPGA has the potential to improve the simulation of cost operation. FPGAs excel at custom circuit design, allowing to create of hardware specifically optimised for bitwise-XOR and PopCount operations. In FPGA, one can design two independent circuits to run in parallel for XOR and population count. Circuits for XOR require two bits in a single clock cycle and can be completed in a few nanoseconds, as they are implemented in hardware and do not require instruction decoding or complex memory hierarchies. Also, FPGA executes the operation deterministically, which could reduce overhead for memory contention and synchronisation during state vector updates.

C. Mixer Hamiltonian

Implementing the mixer operation presents more challenges compared to the cost Hamiltonian, being the main source of computation cost in the simulation. The mixer Hamiltonian is implemented as a transverse-field mixer, mathematically represented as:

$$U_M = e^{-i\beta \sum_i X_i} = \prod_{k=1}^{p} e^{-i\beta_k x_i} \tag{8}$$

which requires applying uniform rotations around the x-axis to induce transitions between the computational basis states or Walsh-Hadamard transformation ([1], [27]) on each qubit to mix the probability amplitudes of the state vector elements.

$$e^{-i\beta X} = \cos(\beta)I - i\sin(\beta)X \tag{9}$$

Classical simulation, especially in the transverse-field mixer, requires efficient communication between all nodes, as each output element depends on all input components, leading to substantial computational overhead. We aim to adopt the GPU-accelerated state vector propagation strategy developed in [20] on FPGA hardware. This method parallelises state vector updates significantly, reducing computational overhead for single-qubit rotations, especially for high-depth circuits. While GPU implementations require extensive resources, FPGA architectures promise greater power efficiency and scalability.

Our approach aims to tackle this shortcoming by extending a similar core concept for implementing QAOA simulation on FPGA. The main objective is to seek whatever FPGA architecture can provide superior speed-up and scalability, and potentially outperform both GPU-accelerated and CPU-based implementations. This transition from GPU to FPGA simulation not only could provide deeper insights into the feasibility of hardware acceleration for quantum-classical hybrid algorithms, aspires to establish hardware supremacy in simulating high-depth QAOA circuits. Fig. 1 demonstrates an overview of simulating the mixer-layer on FPGA. This design employs a hierarchical multi-tier segmentation strategy that maximises parallelism and minimises communication overhead. At the global level, the initial full-size (2^n for n-qubit) state vector is partitioned into different contiguous segments that are independently assigned to separate FPGA regions or memory blocks. Global segmentation ensures that each region contains complete pairs of amplitudes for the single-qubit mixer update, thereby enabling local computations with minimal inter-region data transfers. In each region, local segmentation further breaks the segment into smaller blocks, each managed by a set of pipelined, replicated pair-update modules.

According to the core computation level as mentioned in (9), the mixer operation is decomposed into its real and imaginary components in Fig. 1. In the deeper FPGA layer, dedicated DSP slices perform the constant multiplication, i.e., multiplying the state amplitudes by pre-computed $\cos(\beta)$ and $\sin(\beta)$ values—while the configurable logic blocks (CLBs) implement the adders, subtracters, control logic, and simple bitwise operations such as the sign inversion required for i-multiplication. Extensive pipelining and efficient utilisation of on-chip memory, such as BRAM, enable simultaneous updates of amplitude pairs, enhancing throughput and resource optimisation in FPGA-based QAOA simulation.

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 21:05:09 UTC from IEEE Xplore. Restrictions apply.

IV. CURRENT IMPLEMENTATION STATUS

A. FPGA Architecture Selection

After reviewing the potential FPGA boards, we selected the Terasic TR5 Stratix V GX FPGA Development Kit for its high on-board memory (HBM), superior DSP slice density, extensive on-chip memory capacity, and robust support for highly parallel computation tasks such as QAOA simulation. Additionally, this FPGA model was selected because our university maintains a significant number of these boards interconnected as a parallel computing resource known as Hydra-AI, providing a readily available, scalable environment ideal for developing and benchmarking the FPGA-based QAOA simulation. This state-of-the-art board, featuring numerous DSP slices and CLBs, has made it suitable for our architecture, since it maximises the parallelism in the mixer operation by utilising a multi-tiered segmentation method. Based on the core computation and precision, single-precision (complex-64) is planned for arithmetic offloaded to the FPGAs.

Regarding resource utilisation, preliminary estimates indicate significant use of DSP slices for arithmetic operations, especially complex multiplication and trigonometric evaluations. Memory bandwidth limitations and precision trade-offs have been considered, with single-precision floating-point arithmetic currently preferred to simplify simulation and enhance stability. Fixed-point arithmetic remains an option to reduce resource usage. Potential bottlenecks include limited on-chip memory for large state vectors and overhead from off-chip memory communication, both requiring careful optimisation in future implementations.

B. Hierarchical FPGA Data Flow Architecture

Fig. 2 shows a five-layer hierarchical architecture of an FPGA-based simulation framework. First, the Host and System Control layer handles parameter configuration and dynamic reconfiguration via HLS or embedded controllers. Then, FPGA Fabric Partitioning and Optimised Interconnect layer employs resource-aware floor planning and dedicated routing channels to support high-frequency pipelining. In order to provide effective streaming of state changes and broadcast of precomputed constants, the Parallel Dataflow and Systolic Architecture layer duplicates and pipelines pair update modules in a systolic array. While the deepest layer implements the basic FPGA arithmetic primitives, e.g., optimised multipliers, adders, and bitwise operators for constant multiplications via distributed arithmetic, the core mixer operations are carried out by custom IP blocks optimised for deep pipelining and resource sharing using DSP slices in the Dedicated Hardware Modules layer. This multi-tiered technique makes large-scale QAOA simulations possible while reducing communication overhead.

C. Key Challenges

A primary challenge in simulating the QAOA is partitioning the large-state vector of the mixer operator to maximise parallelism and minimise communication overhead. Also,

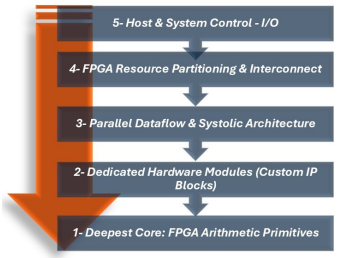

Fig. 2. The hierarchical Data flow of FPGA-based QAOA simulation

optimally balancing the use of DSP slices for multiplication and CLBs for subtraction/addition is critical to utilising the efficiency of the FPGA hardware. Moreover, interconnecting multiple FPGA regions or boards is inherently more complex than numerous libraries for using multiple GPUs, which introduces additional resource management challenges during simulation.

Another critical challenge is managing the high memory demand of the quantum set vector. Given the limited capacity of on-chip memory (e.g., BRAM), it is essential to partition and distribute the state vector data effectively across the FPGA fabric to ensure rapid, parallel access. In addition, access to off-chip or host memory raises latency in FPGA compared to GPU. Therefore, optimising memory bandwidth and minimising access latency is central to achieving scalable and efficient FPGA-based simulations.

V. NEXT STEPS

A. FPGA Networking

In the initial stage of this project, we employ a single-board implementation to perform a small-scale simulation, ensuring that each segment is appropriately processed. We are implementing a single-FPGA architecture including both the mixer and cost operators within the unified read-compute-write pipeline. In this design, proper address generation enables the sequential fetching of state vector data, the application of precomputed functions (e.g., using small lookup tables or CORDIC-like blocks for trigonometric evaluations), and the subsequent writing back of updated data—all within a single streamlined data path. By instantiating multiple parallel "cost update" and "mixer update" units, we aim to fully leverage the FPGA's DSP slices and on-chip memory bandwidth. For functions such as exponentials and trigonometric calculations, pre-computation will significantly lower the logic overhead, reducing the need for both DSP and LUT.

B. Comparison and Performance Evaluation

Once we achieved functional simulation, we will benchmark the FPGA design against CPU/GPU-based simulators by (a) comparing with standard libraries such as Qiskit Aer or Cirq; and (b) using frameworks such as NVIDIA's cuQuantum SDK, JuliQAOA, and QOkit for quantum simulation. Comparative evaluations will focus on execution time, scalability, power efficiency, and computational costs. Preliminary

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 21:05:09 UTC from IEEE Xplore. Restrictions apply.

analysis suggests that our FPGA approach has significant advantages in terms of parallel computation; however, potential bottlenecks such as memory bandwidth constraints and inter-chip communication overhead may emerge at larger scales. These issues will be explicitly studied to identify scalability limits and to propose further enhancements.

C. Testing for Real-world Applications

We will consider cross-domain optimisation scenarios for testing the new QAOA simulation approach, particularly following up the use cases from our previous research works, i.e., power network optimisation that can be reduced to max-cut problem, and fuzzy-quantum systems. For the latter, it is already tested how the complex calculations of a large fuzzy system's inference engine and/or defuzzification block (e.g., in type-2) can be converted to quantum-ready optimisation problems. The next stage would be a comparative analysis of solving these problems on different QAOA simulators.

VI. Conclusion

This paper presented the position of the ongoing research to implement the QAOA simulation framework on FPGA hardware and compare its performance against CPU/GPU-based simulators. Our investigation reveals that although GPU and CPU-based simulation provided significant progress, they are nearing the limits of efficiency for state vector simulations of QAOA due to constraints in scalability, execution time, and energy consumption. To address this limitation, our work focuses on the inherent parallelism of the FPGA-based acceleration to exploit hardware parallelism to accelerate QAOA state vector simulation with lower latency and improve power efficiency. Our initial design explores techniques such as precomputed cost Hamiltonians, Optimised matrix decomposition, and parallelised computation pipelines for efficient FPGA-based quantum circuit simulation.

In future, we will focus on finalising FPGA implementation on single and multiple FPGA boards, benchmarking against CPU/GPU simulators, and extending the simulation framework to tackle real-world combinatorial optimisation problems. We believe that our study will not only advance FPGA-based quantum acceleration but also contribute meaningfully to the broader discussion on classical quantum simulation techniques. Overall, this study opens new avenues for achieving scalable and energy-efficient quantum simulations, paving the way for further innovations in hardware-accelerated quantum algorithms to be used for optimisation scenarios across different application areas.

References

[1] E. Farhi, J. Goldstone, and S. Gutmann, "A quantum approximate optimization algorithm," *arXiv preprint arXiv:1411.4028*, 2014.

[2] A. Chiatto, A. Alizadeh, G. Acampora, A. Vitiello, A. Pourabdollah, and A. Lotfi, "A performance study of genetic algorithms-based quantum approximate optimisation in the context of power networks," in *UKCI Workshop*. Springer, 2024, pp. 303–314.

[3] A. Pourabdollah, G. Acampora, and R. Schiattarella, "Fuzzy logic on quantum annealers," *IEEE Trans on Fuzzy Sys*, vol. 30, no. 8, pp. 3389–3394, 2021.

[4] G. Acampora, R. Schiattarella, and A. Vitiello, "On the implementation of fuzzy inference engines on quantum computers," *IEEE Trans on Fuzzy Sys*, vol. 31, no. 5, pp. 1419–1433, 2022.

[5] A. Pourabdollah, C. Wilmott, R. Schiattarella, and G. Acampora, "Fuzzy inference on quantum annealers," in *IEEE Intl Conf on Fuzzy Sys*. IEEE, 2023, pp. 1–6.

[6] A. Pourabdollah and J. M. Mendel, "A non-iterative quantum computation for Karnik-Mendel algorithms," in *IEEE Intl Conf on Fuzzy Sys*. IEEE, 2024, pp. 1–6.

[7] S. H. Sack and D. J. Egger, "Large-scale quantum approximate optimization on nonplanar graphs with machine learning noise mitigation," *Physical Review Research*, vol. 6, no. 1, p. 013223, 2024.

[8] H. Shang, Y. Fan, L. Shen, C. Guo, J. Liu, X. Duan, F. Li, and Z. Li, "Towards practical and massively parallel quantum computing emulation for quantum chemistry," *NPJ Quantum Inf*, vol. 9, no. 1, p. 33, 2023.

[9] A. Mahram and M. C. Herbordt, "NCBI BLASTP on high-performance reconfigurable computing systems," *ACM Trans on Reconfigurable Technology Systems*, vol. 7, no. 4, pp. 1–20, 2015.

[10] I. Belfore and A. Lee, "A scalable FPGA architecture for quantum computing simulation," *arXiv preprint arXiv:2407.06415*, 2024.

[11] H. Li and Y. Pang, "FPGA-accelerated quantum computing emulation quantum key distillation," *IEEE Micro*, vol. 41, no. 4, pp. 49–57, 2021.

[12] A. Cicero, M. A. Maleki, M. W. Azhar, A. F. Kockum, and P. Trancoso, "Simulation of quantum computers: Review and acceleration opportunities," *arXiv preprint arXiv:2410.12660*, 2024.

[13] C.-Y. Liu, H. M. Waidyasooriya, and M. Hariyama, "Design space exploration for an FPGA-based quantum annealing simulator with interaction-coefficient-generators," *J. of Supercomputing*, vol. 78, no. 1, pp. 1–17, 2022.

[14] A. Lucas, "Ising formulations of many NP problems," *Frontiers in physics*, vol. 2, p. 5, 2014.

[15] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," *SIAM review*, vol. 41, no. 2, pp. 303–332, 1999.

[16] B. Zhang, B. Fang, F. Ye, Y. Gu, N. Tallent, G. Tan, and D. Tao, "Overcoming memory constraints in quantum circuit simulation with a high-fidelity compression framework," *arXiv preprint 2410.14088*, 2024.

[17] D. Lykov, "Large-scale tensor network quantum algorithm simulator," Master's thesis, Chicago Univ., 2024.

[18] X.-C. Wu, S. Di, E. M. Dasgupta, F. Cappello, H. Finkel, Y. Alexeev, and F. T. Chong, "Full-state quantum circuit simulation by using data compression," in *Proc. of Intl Conf on HPC, Networking, Storage Analysis*, 2019, pp. 1–24.

[19] S. V. Isakov *et al.*, "Simulations of quantum circuits with approximate noise using qsim and Cirq," *arXiv preprint arXiv:2111.02396*, 2021.

[20] D. Lykov, R. Shaydulin, Y. Sun, Y. Alexeev, and M. Pistoia, "Fast simulation of high-depth QAOA circuits," in *Proc of the Intl Conf on HPC, Network, Storage Analysis*, 2023, pp. 1443–1451.

[21] J. Golden, A. Baertschi, D. O'Malley, E. Pelofske, and S. Eidenbenz, "JuliQAOA: Fast, flexible QAOA simulation," in *Proc of the Intl Conf on HPC, Network, Storage Analysis*, 2023, pp. 1454–1459.

[22] J. Bezanson, A. Edelman, S. Karpinski, and V. B. Shah, "Julia: A fresh approach to numerical computing," *SIAM review*, vol. 59, no. 1, pp. 65–98, 2017.

[23] H. Bayraktar *et al.*, "cuQuantum SDK: A high-performance library for accelerating quantum science," in *Intl Conf on Quantum Comp and Eng*, vol. 1. IEEE, 2023, pp. 1050–1061.

[24] J. Stein, J. Blenninger, D. Bucher, P. J. Eder, E. Çetiner, M. Zorn, and C. Linnhoff-Popien, "CUAOA: A novel CUDA-accelerated simulation framework for the QAOA," in *Intl Conf on Quantum Comp and Eng*, vol. 2. IEEE, 2024, pp. 280–285.

[25] S. Tibaldi, D. Vodola, E. Tignone, and E. Ercolessi, "Bayesian optimization for qaoa," *arXiv preprint arXiv:2209.03824*, 2023.

[26] L. Cheng, Y.-Q. Chen, S.-X. Zhang, and S. Zhang, "Quantum approximate optimization via learning-based adaptive optimization," *Communications Physics*, vol. 7, no. 83, 2024.

[27] T. Hogg, "Quantum search heuristics," *Physical Review A*, vol. 61, no. 5, p. 052311, 2000.

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 21:05:09 UTC from IEEE Xplore. Restrictions apply.